Zipcode Project 3

Generated by Doxygen 1.16.1

1 Class Index 1
1.1 Class List . . . 1

2 File Index 3
2.1 File List . . . 3

3 Class Documentation 5
3.1 BlockBuffer Class Reference . . . 5
3.1.1 Detailed Description . . . 6
3.1.2 Constructor & Destructor Documentation . . . 7
3.1.2.1 BlockBuffer() . . . 7
3.1.3 Member Function Documentation . . . 7
3.1.3.1 canFit() . . . 7
3.1.3.2 clearToAvail() . . . 7
3.1.3.3 findRecord() . . . 8
3.1.3.4 getBlockSize() . . . 8
3.1.3.5 getFreeBytes() . . . 8
3.1.3.6 getHighestKey() . . . 8
3.1.3.7 getNextRBN() . . . 9
3.1.3.8 getPrevRBN() . . . 9
3.1.3.9 getRecordCount() . . . 9
3.1.3.10 getRecords() . . . 9
3.1.3.11 getUsedBytes() . . . 9
3.1.3.12 insertRecordSorted() . . . 10
3.1.3.13 isActiveBlock() . . . 10
3.1.3.14 isAvailBlock() . . . 10
3.1.3.15 pack() . . . 10
3.1.3.16 removeRecordByZip() . . . 11
3.1.3.17 setNextRBN() . . . 11
3.1.3.18 setPrevRBN() . . . 11
3.1.3.19 unpack() . . . 11
3.2 BlockedFileHeader Class Reference . . . 12
3.2.1 Detailed Description . . . 13
3.2.2 Member Data Documentation . . . 13
3.2.2.1 activeListHead . . . 13
3.2.2.2 availListHead . . . 13
3.2.2.3 blockCount . . . 13
3.2.2.4 blockSize . . . 13
3.2.2.5 fieldCount . . . 14
3.2.2.6 fields . . . 14
3.2.2.7 fileStructureType . . . 14
3.2.2.8 headerSize . . . 14
3.2.2.9 indexFileStructureType . . . 14

3.2.2.10 minBlockCapacity . . . 14
3.2.2.11 primaryKeyFieldIndex . . . 14
3.2.2.12 primaryKeyIndexFileName . . . 14
3.2.2.13 recordCount . . . 15
3.2.2.14 recordSizeIntegerBytes . . . 15
3.2.2.15 sizeFormatType . . . 15
3.2.2.16 sizeInclusionFlag . . . 15
3.2.2.17 sizeOfSizes . . . 15
3.2.2.18 staleFlag . . . 15
3.2.2.19 version . . . 15
3.3 BlockFileBuffer Class Reference . . . 16
3.3.1 Detailed Description . . . 16
3.3.2 Constructor & Destructor Documentation . . . 17
3.3.2.1 BlockFileBuffer() . . . 17
3.3.3 Member Function Documentation . . . 17
3.3.3.1 getBlockOffset() . . . 17
3.3.3.2 getFilename() . . . 17
3.3.3.3 isReadOpen() . . . 17
3.3.3.4 isWriteOpen() . . . 18
3.3.3.5 openForRead() . . . 18
3.3.3.6 openForUpdate() . . . 18
3.3.3.7 openForWrite() . . . 18
3.3.3.8 readBlock() . . . 19
3.3.3.9 readHeader() . . . 19
3.3.3.10 writeBlock() . . . 19
3.3.3.11 writeHeader() . . . 20
3.4 BlockIndex Class Reference . . . 20
3.4.1 Detailed Description . . . 21
3.4.2 Constructor & Destructor Documentation . . . 21
3.4.2.1 BlockIndex() . . . 21
3.4.3 Member Function Documentation . . . 22
3.4.3.1 addEntry() . . . 22
3.4.3.2 findCandidateBlock() . . . 22
3.4.3.3 getEntries() . . . 22
3.4.3.4 getFilename() . . . 23
3.4.3.5 loadIndex() . . . 23
3.4.3.6 size() . . . 23
3.4.3.7 writeIndex() . . . 23
3.5 BlockIndexEntry Struct Reference . . . 24
3.5.1 Detailed Description . . . 24
3.5.2 Member Data Documentation . . . 24
3.5.2.1 maxKey . . . 24

3.5.2.2 rbn . . . 24
3.6 DataFileHeader Class Reference . . . 24
3.6.1 Detailed Description . . . 25
3.6.2 Constructor & Destructor Documentation . . . 26
3.6.2.1 DataFileHeader() [1/2] . . . 26
3.6.2.2 DataFileHeader() [2/2] . . . 26
3.6.3 Member Function Documentation . . . 27
3.6.3.1 addField() . . . 27
3.6.3.2 display() . . . 27
3.6.3.3 getPackedSize() . . . 27
3.6.3.4 pack() . . . 28
3.6.3.5 unpack() . . . 28
3.6.4 Member Data Documentation . . . 29
3.6.4.1 fieldCount . . . 29
3.6.4.2 fields . . . 29
3.6.4.3 fileStructureType . . . 29
3.6.4.4 headerSize . . . 29
3.6.4.5 primaryKeyFieldIndex . . . 29
3.6.4.6 primaryKeyIndexFileName . . . 29
3.6.4.7 recordCount . . . 29
3.6.4.8 recordSizeIntegerBytes . . . 30
3.6.4.9 sizeFormatType . . . 30
3.6.4.10 sizeInclusionFlag . . . 30
3.6.4.11 sizeOfSizes . . . 30
3.6.4.12 version . . . 30
3.7 FieldDescriptor Struct Reference . . . 30
3.7.1 Detailed Description . . . 31
3.7.2 Constructor & Destructor Documentation . . . 31
3.7.2.1 FieldDescriptor() [1/2] . . . 31
3.7.2.2 FieldDescriptor() [2/2] . . . 31
3.7.3 Member Data Documentation . . . 32
3.7.3.1 isPrimaryKey . . . 32
3.7.3.2 name . . . 32
3.7.3.3 typeSchema . . . 32
3.8 HeaderBuffer Class Reference . . . 32
3.8.1 Detailed Description . . . 33
3.8.2 Constructor & Destructor Documentation . . . 33
3.8.2.1 HeaderBuffer() . . . 33
3.8.2.2 ~HeaderBuffer() . . . 34
3.8.3 Member Function Documentation . . . 34
3.8.3.1 readHeader() . . . 34
3.8.3.2 writeHeader() . . . 35

3.9 LIBuffer Class Reference . . . 36
3.9.1 Detailed Description . . . 36
3.9.2 Constructor & Destructor Documentation . . . 37
3.9.2.1 LIBuffer() . . . 37
3.9.2.2 ~LIBuffer() . . . 38
3.9.3 Member Function Documentation . . . 39
3.9.3.1 close() . . . 39
3.9.3.2 getCurrentOffset() . . . 39
3.9.3.3 getFilename() . . . 40
3.9.3.4 getNextRecord() . . . 40
3.9.3.5 getRecordCount() . . . 41
3.9.3.6 openForRead() . . . 41
3.9.3.7 openForWrite() . . . 42
3.9.3.8 readHeader() . . . 42
3.9.3.9 readRecordAt() . . . 43
3.9.3.10 writeHeader() . . . 44
3.9.3.11 writeRecord() . . . 45
3.10 RecordBuffer Class Reference . . . 46
3.10.1 Detailed Description . . . 47
3.10.2 Member Function Documentation . . . 47
3.10.2.1 buildPayload() . . . 47
3.10.2.2 getPackedSize() . . . 48
3.10.2.3 pack() . . . 48
3.10.2.4 readLength() . . . 49
3.10.2.5 unpack() . . . 49
3.10.2.6 writeLength() . . . 50
3.11 ZipCodeRecord Struct Reference . . . 50
3.11.1 Detailed Description . . . 51
3.11.2 Constructor & Destructor Documentation . . . 51
3.11.2.1 ZipCodeRecord() [1/2] . . . 51
3.11.2.2 ZipCodeRecord() [2/2] . . . 51
3.11.3 Member Function Documentation . . . 52
3.11.3.1 display() . . . 52
3.11.3.2 displayInline() . . . 52
3.11.3.3 getPackedSize() . . . 52

4 File Documentation **53**
4.1 add_seqset.cpp File Reference . . . 53
4.1.1 Detailed Description . . . 54
4.1.2 Function Documentation . . . 54
4.1.2.1 addOneRecord() . . . 54
4.1.2.2 loadAddRecords() . . . 54

4.1.2.3 main() . . . 55
4.1.2.4 parseAddLine() . . . 55
4.1.2.5 printRecord() . . . 55
4.1.2.6 rebuildBlockIndex() . . . 56
4.2 BlockBuffer.cpp File Reference . . . 56
4.2.1 Detailed Description . . . 56
4.3 BlockBuffer.h File Reference . . . 56
4.3.1 Detailed Description . . . 57
4.4 BlockBuffer.h . . . 58
4.5 BlockedFileHeader.cpp File Reference . . . 58
4.5.1 Detailed Description . . . 59
4.6 BlockedFileHeader.h File Reference . . . 59
4.6.1 Detailed Description . . . 59
4.7 BlockedFileHeader.h . . . 60
4.8 BlockFileBuffer.cpp File Reference . . . 60
4.8.1 Detailed Description . . . 61
4.9 BlockFileBuffer.h File Reference . . . 61
4.9.1 Detailed Description . . . 61
4.10 BlockFileBuffer.h . . . 62
4.11 BlockIndex.cpp File Reference . . . 62
4.11.1 Detailed Description . . . 63
4.12 BlockIndex.h File Reference . . . 63
4.12.1 Detailed Description . . . 63
4.13 BlockIndex.h . . . 64
4.14 build_block_index.cpp File Reference . . . 64
4.14.1 Detailed Description . . . 65
4.15 build_seqset.cpp File Reference . . . 65
4.15.1 Detailed Description . . . 65
4.16 DataFileHeader.cpp File Reference . . . 66
4.16.1 Detailed Description . . . 66
4.17 DataFileHeader.h File Reference . . . 66
4.17.1 Detailed Description . . . 66
4.18 DataFileHeader.h . . . 67
4.19 delete_seqset.cpp File Reference . . . 67
4.19.1 Detailed Description . . . 68
4.20 dump_seqset.cpp File Reference . . . 68
4.20.1 Detailed Description . . . 69
4.21 HeaderBuffer.cpp File Reference . . . 69
4.21.1 Detailed Description . . . 69
4.22 HeaderBuffer.h File Reference . . . 70
4.22.1 Detailed Description . . . 70
4.23 HeaderBuffer.h . . . 70

4.24 LIBuffer.cpp File Reference 71
4.24.1 Detailed Description 71
4.25 LIBuffer.h File Reference 71
4.25.1 Detailed Description 72
4.26 LIBuffer.h 72
4.27 RecordBuffer.cpp File Reference 73
4.27.1 Detailed Description 73
4.28 RecordBuffer.h File Reference 74
4.28.1 Detailed Description 74
4.29 RecordBuffer.h 75
4.30 search_seqset.cpp File Reference 75
4.30.1 Detailed Description 76
4.31 ZipCodeBuffer.cpp File Reference 76
4.31.1 Detailed Description 76
4.32 ZipCodeRecord.h File Reference 76
4.32.1 Detailed Description 77
4.33 ZipCodeRecord.h 77

Index **79**

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

BlockBuffer
In-memory representation of one physical block 5
BlockedFileHeader
Stores metadata for a blocked sequence-set file 12
BlockFileBuffer
Provides physical file access for blocked sequence-set files 16
BlockIndex
In-memory simple index for the blocked sequence-set file 20
BlockIndexEntry
One simple block-index entry 24
DataFileHeader
Holds metadata about a length-indicated ZIP code data file 24
FieldDescriptor
Describes a single field in each data record 30
HeaderBuffer
Writes and reads a DataFileHeader as the first record of a .li file 32
LIBuffer
Buffered reader/writer for length-indicated ZIP code data files 36
RecordBuffer
Converts ZipCodeRecord objects to/from raw bytes for block storage 46
ZipCodeRecord
Structure to hold information for a single ZIP code This structure contains all fields from the US Postal Code database: 50

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

add_seqset.cpp
Adds ZIP code records to the GP 3.0 sequence-set file (supports insert without split and insert with split) . . . 53
BlockBuffer.cpp
Implementation of BlockBuffer . . . 56
BlockBuffer.h
Represents one fixed-size block in the blocked sequence-set file . . . 56
BlockedFileHeader.cpp
Implementation of BlockedFileHeader . . . 58
BlockedFileHeader.h
Header structure for the GP 3.0 blocked sequence-set ZIP code file . . . 59
BlockFileBuffer.cpp
Implementation of BlockFileBuffer . . . 60
BlockFileBuffer.h
File I/O layer for blocked sequence-set files . . . 61
BlockIndex.cpp
Implementation of BlockIndex . . . 62
BlockIndex.h
Simple primary-key block index for GP 3.0 . . . 63
build_block_index.cpp
Builds the GP 3.0 simple block index from the sequence-set file . . . 64
build_seqset.cpp
Builds the GP 3.0 blocked sequence-set file from a GP 2.0 LI file . . . 65
DataFileHeader.cpp
Implementation of DataFileHeader . . . 66
DataFileHeader.h
Data structure for the header record of a length-indicated ZIP code file . . . 66
delete_seqset.cpp
Deletes ZIP code records from the GP 3.0 sequence-set file (supports simple delete, redistribution, and merge) . . . 67
dump_seqset.cpp
Dumps a GP 3.0 blocked sequence-set file . . . 68
HeaderBuffer.cpp
Implementation of HeaderBuffer class . . . 69
HeaderBuffer.h
Reads and writes the DataFileHeader as the first length-indicated record of a .li binary data file 70

LIBuffer.cpp
Implementation of LIBuffer class 71
LIBuffer.h
Reads and writes length-indicated ZIP code data files 71
RecordBuffer.cpp
Implementation of RecordBuffer 73
RecordBuffer.h
Packs and unpacks one ZIP code record for blocked sequence-set storage 74
search_seqset.cpp
Indexed ZIP code search for the GP 3.0 sequence-set file 75
ZipCodeBuffer.cpp
Implementation of the ZipCodeBuffer class 76
ZipCodeRecord.h
Data structure for US Postal Code records 76

Chapter 3

Class Documentation

3.1 BlockBuffer Class Reference

In-memory representation of one physical block.

```
#include <BlockBuffer.h>
```

Public Member Functions

- BlockBuffer (int blockSize=512)

  Construct a block with the given fixed block size.
- void **clearToEmptyActive** ()

  Reset this block to an empty active block.
- void clearToAvail (int nextAvailRBN=-1)

  Reset this block to an avail block.
- bool isAvailBlock () const

  Return true if this block is an avail block.
- bool isActiveBlock () const

  Return true if this block is an active block.
- int getBlockSize () const

  Return the fixed physical block size.
- int getRecordCount () const

  Return the number of records stored in this block.
- void setPrevRBN (int rbn)

  Set previous active-block RBN.
- void setNextRBN (int rbn)

  Set next active-block RBN.
- int getPrevRBN () const

  Get previous active-block RBN.
- int getNextRBN () const

  Get next active-block RBN.
- size_t getUsedBytes () const

  Return number of bytes used inside this block.
- size_t getFreeBytes () const

  Return number of bytes still free in this block.

- bool canFit (const ZipCodeRecord &record) const

  Return true if a record can fit into the block.
- bool insertRecordSorted (const ZipCodeRecord &record)

  Insert a record in ZIP-sorted order.
- bool removeRecordByZip (int zipCode)

  Remove a record by ZIP code.
- bool findRecord (int zipCode, ZipCodeRecord &record) const

  Search for a record by ZIP code.
- int getHighestKey () const

  Return the highest ZIP key in the block.
- const std::vector< ZipCodeRecord > & getRecords () const

  Return all records in this block.
- bool pack (char *buffer) const

  Pack the full block image into a raw byte buffer.
- bool unpack (const char *buffer)

  Unpack a full block image from a raw byte buffer.
- void **display** () const

  Print block contents for debugging.

3.1.1 Detailed Description

In-memory representation of one physical block.

Responsibilities:

- store active-block metadata
- store avail-block metadata
- keep records sorted by ZIP code
- pack/unpack the full fixed-size block image
- calculate used/free bytes

NOT responsible for:

- file I/O
- global sequence-set management
- index creation

3.1.2 Constructor & Destructor Documentation

3.1.2.1 BlockBuffer()

```
BlockBuffer::BlockBuffer (
            int blockSize = 512)  [explicit]
```

Construct a block with the given fixed block size.

Parameters

blockSize	Number of bytes in the physical block.

3.1.3 Member Function Documentation

3.1.3.1 canFit()

```
bool BlockBuffer::canFit (
            const ZipCodeRecord & record) const
```

Return true if a record can fit into the block.

Parameters

record	Record to test.

Returns

true if there is enough free space.

3.1.3.2 clearToAvail()

```
void BlockBuffer::clearToAvail (
            int nextAvailRBN = -1)
```

Reset this block to an avail block.

Parameters

nextAvailRBN	Link to next avail block; -1 if none.

3.1.3.3 findRecord()

```
bool BlockBuffer::findRecord (
            int zipCode,
            ZipCodeRecord & record) const
```

Search for a record by ZIP code.

Parameters

zipCode	ZIP code to find.
record	Output record if found.

Returns

true if found; false otherwise.

3.1.3.4 getBlockSize()

```
int BlockBuffer::getBlockSize () const
```

Return the fixed physical block size.

Returns

Block size in bytes.

3.1.3.5 getFreeBytes()

```
size_t BlockBuffer::getFreeBytes () const
```

Return number of bytes still free in this block.

Returns

Free byte count.

3.1.3.6 getHighestKey()

```
int BlockBuffer::getHighestKey () const
```

Return the highest ZIP key in the block.

Returns

Highest ZIP code, or -1 if no active records.

3.1.3.7 getNextRBN()

```
int BlockBuffer::getNextRBN () const
```

Get next active-block RBN.

Returns

Next RBN.

3.1.3.8 getPrevRBN()

```
int BlockBuffer::getPrevRBN () const
```

Get previous active-block RBN.

Returns

Previous RBN.

3.1.3.9 getRecordCount()

```
int BlockBuffer::getRecordCount () const
```

Return the number of records stored in this block.

Returns

Number of active records.

3.1.3.10 getRecords()

```
const std::vector< ZipCodeRecord > & BlockBuffer::getRecords () const
```

Return all records in this block.

Returns

Const reference to sorted record vector.

3.1.3.11 getUsedBytes()

```
size_t BlockBuffer::getUsedBytes () const
```

Return number of bytes used inside this block.

Returns

Used byte count.

3.1.3.12 insertRecordSorted()

```
bool BlockBuffer::insertRecordSorted (
            const ZipCodeRecord & record)
```

Insert a record in ZIP-sorted order.

Parameters

record	Record to insert.

Returns

true if inserted; false if duplicate ZIP or insufficient space.

3.1.3.13 isActiveBlock()

```
bool BlockBuffer::isActiveBlock () const
```

Return true if this block is an active block.

Returns

true if recordCount $>$ 0.

3.1.3.14 isAvailBlock()

```
bool BlockBuffer::isAvailBlock () const
```

Return true if this block is an avail block.

Returns

true if recordCount == 0.

3.1.3.15 pack()

```
bool BlockBuffer::pack (
            char * buffer) const
```

Pack the full block image into a raw byte buffer.

Parameters

buffer	Caller-allocated buffer of exactly blockSize bytes.

Returns

true on success; false if packed data exceeds block size.

3.1.3.16 removeRecordByZip()

```
bool BlockBuffer::removeRecordByZip (
            int zipCode)
```

Remove a record by ZIP code.

Parameters

zipCode	ZIP code to remove.

Returns

true if removed; false if not found.

3.1.3.17 setNextRBN()

```
void BlockBuffer::setNextRBN (
            int rbn)
```

Set next active-block RBN.

Parameters

rbn	Next active block number.

3.1.3.18 setPrevRBN()

```
void BlockBuffer::setPrevRBN (
            int rbn)
```

Set previous active-block RBN.

Parameters

rbn	Previous active block number.

3.1.3.19 unpack()

```
bool BlockBuffer::unpack (
            const char * buffer)
```

Unpack a full block image from a raw byte buffer.

Parameters

buffer	Input buffer of exactly blockSize bytes.

Returns

true on success; false on malformed input.

The documentation for this class was generated from the following files:

- BlockBuffer.h
- BlockBuffer.cpp

3.2 BlockedFileHeader Class Reference

Stores metadata for a blocked sequence-set file.

```
#include <BlockedFileHeader.h>
```

Public Member Functions

- **BlockedFileHeader** (const std::string &structType, int ver)
- void **addField** (const std::string &name, const std::string &typeSchema, bool isPrimaryKey=false)
- size_t **getPackedSize** () const
- size_t **pack** (char ∗buffer) const
- size_t **unpack** (const char ∗buffer)
- void **display** () const

Public Attributes

- std::string fileStructureType
- int version
- int headerSize
- int recordSizeIntegerBytes
- std::string sizeFormatType
- int sizeOfSizes
- bool sizeInclusionFlag
- int blockSize
- double minBlockCapacity
- int recordCount
- int blockCount
- int activeListHead
- int availListHead
- bool staleFlag
- std::string primaryKeyIndexFileName
- std::string indexFileStructureType
- int primaryKeyFieldIndex
- int fieldCount
- std::vector< FieldDescriptor > fields

3.2.1 Detailed Description

Stores metadata for a blocked sequence-set file.

Responsibilities:

- store file-level metadata
- store field descriptions
- pack/unpack itself into a raw buffer
- display itself for debugging

NOT responsible for:

- opening or closing files
- reading/writing streams directly

3.2.2 Member Data Documentation

3.2.2.1 activeListHead

```
int BlockedFileHeader::activeListHead
```

RBN of first active block; -1 if none

3.2.2.2 availListHead

```
int BlockedFileHeader::availListHead
```

RBN of first avail block; -1 if none

3.2.2.3 blockCount

```
int BlockedFileHeader::blockCount
```

Total physical blocks in file

3.2.2.4 blockSize

```
int BlockedFileHeader::blockSize
```

Physical bytes per block

3.2.2.5 fieldCount

```
int BlockedFileHeader::fieldCount
```

Number of fields in each record

3.2.2.6 fields

```
std::vector<FieldDescriptor> BlockedFileHeader::fields
```

Field descriptors

3.2.2.7 fileStructureType

```
std::string BlockedFileHeader::fileStructureType
```

e.g. "ZIPCODE_BSS_V1"

3.2.2.8 headerSize

```
int BlockedFileHeader::headerSize
```

Packed header size

3.2.2.9 indexFileStructureType

```
std::string BlockedFileHeader::indexFileStructureType
```

e.g. "ZIPCODE_BLOCK_INDEX_V1"

3.2.2.10 minBlockCapacity

```
double BlockedFileHeader::minBlockCapacity
```

Minimum occupancy ratio (0.50 default)

3.2.2.11 primaryKeyFieldIndex

```
int BlockedFileHeader::primaryKeyFieldIndex
```

Which field is the PK

3.2.2.12 primaryKeyIndexFileName

```
std::string BlockedFileHeader::primaryKeyIndexFileName
```

Associated block-index filename

3.2.2.13 recordCount

```
int BlockedFileHeader::recordCount
```

Total active records in file

3.2.2.14 recordSizeIntegerBytes

```
int BlockedFileHeader::recordSizeIntegerBytes
```

Bytes used for LI size fields

3.2.2.15 sizeFormatType

```
std::string BlockedFileHeader::sizeFormatType
```

e.g. "binary"

3.2.2.16 sizeInclusionFlag

```
bool BlockedFileHeader::sizeInclusionFlag
```

Whether size includes itself

3.2.2.17 sizeOfSizes

```
int BlockedFileHeader::sizeOfSizes
```

Size of size fields

3.2.2.18 staleFlag

```
bool BlockedFileHeader::staleFlag
```

True if index is out-of-date

3.2.2.19 version

```
int BlockedFileHeader::version
```

File format version

The documentation for this class was generated from the following files:

- BlockedFileHeader.h
- BlockedFileHeader.cpp

3.3 BlockFileBuffer Class Reference

Provides physical file access for blocked sequence-set files.

```
#include <BlockFileBuffer.h>
```

Public Member Functions

- BlockFileBuffer (const std::string &filename)

  Construct a BlockFileBuffer for a filename.
- ~**BlockFileBuffer** ()

  Destructor.
- bool openForRead ()

  Open file for reading.
- bool openForWrite ()

  Open file for writing, truncating old contents.
- bool openForUpdate ()

  Open file for read/write update without truncating.
- void **close** ()

  Close any open file streams.
- bool writeHeader (BlockedFileHeader &header)

  Write the blocked-file header at the start of the file.
- bool readHeader (BlockedFileHeader &header)

  Read the blocked-file header from the start of the file.
- bool writeBlock (int rbn, const BlockBuffer &block)

  Write one physical block by RBN.
- bool readBlock (int rbn, BlockBuffer &block)

  Read one physical block by RBN.
- std::streamoff getBlockOffset (int rbn, int blockSize) const

  Return the byte offset of a given block.
- std::string getFilename () const

  Return the filename associated with this object.
- bool isReadOpen () const

  Return true if file is currently open for reading.
- bool isWriteOpen () const

  Return true if file is currently open for writing/updating.

3.3.1 Detailed Description

Provides physical file access for blocked sequence-set files.

3.3.2 Constructor & Destructor Documentation

3.3.2.1 BlockFileBuffer()

```
BlockFileBuffer::BlockFileBuffer (
            const std::string & filename)  [explicit]
```

Construct a BlockFileBuffer for a filename.

Parameters

filename	Path to blocked data file.

3.3.3 Member Function Documentation

3.3.3.1 getBlockOffset()

```
std::streamoff BlockFileBuffer::getBlockOffset (
            int rbn,
            int blockSize) const
```

Return the byte offset of a given block.

Parameters

rbn	Relative block number.
blockSize	Size of each physical block in bytes.

Returns

Absolute byte offset in the file.

3.3.3.2 getFilename()

```
std::string BlockFileBuffer::getFilename () const
```

Return the filename associated with this object.

Returns

Filename string.

3.3.3.3 isReadOpen()

```
bool BlockFileBuffer::isReadOpen () const
```

Return true if file is currently open for reading.

Returns

true if readable stream is open.

3.3.3.4 isWriteOpen()

```
bool BlockFileBuffer::isWriteOpen () const
```

Return true if file is currently open for writing/updating.

Returns

true if writable stream is open.

3.3.3.5 openForRead()

```
bool BlockFileBuffer::openForRead ()
```

Open file for reading.

Returns

true on success; false otherwise.

3.3.3.6 openForUpdate()

```
bool BlockFileBuffer::openForUpdate ()
```

Open file for read/write update without truncating.

Returns

true on success; false otherwise.

3.3.3.7 openForWrite()

```
bool BlockFileBuffer::openForWrite ()
```

Open file for writing, truncating old contents.

Returns

true on success; false otherwise.

3.3.3.8 readBlock()

```
bool BlockFileBuffer::readBlock (
            int rbn,
            BlockBuffer & block)
```

Read one physical block by RBN.

Parameters

rbn	Relative block number (0-based).
block	Output block object.

Returns

true on success; false otherwise.

3.3.3.9 readHeader()

```
bool BlockFileBuffer::readHeader (
            BlockedFileHeader & header)
```

Read the blocked-file header from the start of the file.

Parameters

header	Output header.

Returns

true on success; false otherwise.

Note

This stores the end-of-header byte position internally so that future block offsets can be calculated correctly.

3.3.3.10 writeBlock()

```
bool BlockFileBuffer::writeBlock (
            int rbn,
            const BlockBuffer & block)
```

Write one physical block by RBN.

Parameters

rbn	Relative block number (0-based).

block	Block to write.

Returns

true on success; false otherwise.

3.3.3.11 writeHeader()

```
bool BlockFileBuffer::writeHeader (
            BlockedFileHeader & header)
```

Write the blocked-file header at the start of the file.

Parameters

header	Header to write.

Returns

true on success; false otherwise.

Note

This stores the end-of-header byte position internally so that future block offsets can be calculated correctly.

The documentation for this class was generated from the following files:

- BlockFileBuffer.h
- BlockFileBuffer.cpp

3.4 BlockIndex Class Reference

In-memory simple index for the blocked sequence-set file.

```
#include <BlockIndex.h>
```

Public Member Functions

- BlockIndex (const std::string &indexFilename)

  Construct an index associated with a filename.
- void **clear** ()

  Clear all in-memory entries.
- void addEntry (int maxKey, int rbn)

  Add one entry to the in-memory index.
- void **sortEntries** ()

  Sort entries by maxKey ascending.
- bool writeIndex () const

  Write index entries to the binary index file.
- bool loadIndex ()

  Load index entries from the binary index file.
- bool findCandidateBlock (int zipCode, int &rbn) const

  Find the candidate block for a target ZIP code.
- size_t size () const

  Return the number of entries in memory.
- std::string getFilename () const

  Return the associated index filename.
- const std::vector< BlockIndexEntry > & getEntries () const

  Return const reference to all entries.
- void **display** () const

  Print the index in readable form.

3.4.1 Detailed Description

In-memory simple index for the blocked sequence-set file.

Responsibilities:

- store block index entries
- sort entries
- write/read binary index file
- find candidate block for a ZIP code via binary search

3.4.2 Constructor & Destructor Documentation

3.4.2.1 BlockIndex()

```
BlockIndex::BlockIndex (
            const std::string & indexFilename)  [explicit]
```

Construct an index associated with a filename.

Parameters

indexFilename	Path to binary block-index file.

3.4.3 Member Function Documentation

3.4.3.1 addEntry()

```
void BlockIndex::addEntry (
            int maxKey,
            int rbn)
```

Add one entry to the in-memory index.

Parameters

maxKey	Highest ZIP in the block.
rbn	Relative block number.

3.4.3.2 findCandidateBlock()

```
bool BlockIndex::findCandidateBlock (
            int zipCode,
            int & rbn) const
```

Find the candidate block for a target ZIP code.

Parameters

zipCode	ZIP code being searched.
rbn	Output candidate RBN.

Returns

true if a candidate block exists; false otherwise.

Uses lower_bound on maxKey values to find the first block whose maxKey >= zipCode.

3.4.3.3 getEntries()

```
const std::vector< BlockIndexEntry > & BlockIndex::getEntries () const
```

Return const reference to all entries.

Returns

Const vector reference.

3.4.3.4 getFilename()

```
std::string BlockIndex::getFilename () const
```

Return the associated index filename.

Returns

Filename string.

3.4.3.5 loadIndex()

```
bool BlockIndex::loadIndex ()
```

Load index entries from the binary index file.

Returns

true on success; false otherwise.

3.4.3.6 size()

```
size_t BlockIndex::size () const
```

Return the number of entries in memory.

Returns

entries.size()

3.4.3.7 writeIndex()

```
bool BlockIndex::writeIndex () const
```

Write index entries to the binary index file.

Returns

true on success; false otherwise.

The documentation for this class was generated from the following files:

- BlockIndex.h
- BlockIndex.cpp

3.5 BlockIndexEntry Struct Reference

One simple block-index entry.

```
#include <BlockIndex.h>
```

Public Member Functions

- **BlockIndexEntry** (int key, int blockNum)
- bool **operator**< (const BlockIndexEntry &other) const

Public Attributes

- int32_t maxKey
- int32_t rbn

3.5.1 Detailed Description

One simple block-index entry.

3.5.2 Member Data Documentation

3.5.2.1 maxKey

```
int32_t BlockIndexEntry::maxKey
```

Highest ZIP key stored in the block

3.5.2.2 rbn

```
int32_t BlockIndexEntry::rbn
```

Relative block number

The documentation for this struct was generated from the following file:

- BlockIndex.h

3.6 DataFileHeader Class Reference

Holds metadata about a length-indicated ZIP code data file.

```
#include <DataFileHeader.h>
```

Public Member Functions

- DataFileHeader ()

  Default constructor — initialises all fields to safe defaults.
- DataFileHeader (const std::string &structType, int ver)

  Parameterised constructor.
- void addField (const std::string &name, const std::string &typeSchema, bool isPrimaryKey=false)

  Append a field descriptor and update fieldCount.
- size_t getPackedSize () const

  Calculate the number of bytes needed to pack this header.
- size_t pack (char *buffer) const

  Serialise this header into a raw byte buffer.
- size_t unpack (const char *buffer)

  Deserialise this header from a raw byte buffer.
- void display () const

  Print all header fields to std::cout in a formatted layout.

Public Attributes

- std::string fileStructureType
- int version
- int headerSize
- int recordSizeIntegerBytes
- std::string sizeFormatType
- int sizeOfSizes
- bool sizeInclusionFlag
- std::string primaryKeyIndexFileName
- int recordCount
- int fieldCount
- int primaryKeyFieldIndex
- std::vector< FieldDescriptor > fields

3.6.1 Detailed Description

Holds metadata about a length-indicated ZIP code data file.

Responsibilities:

- Store file-level metadata (type, version, record count, etc.)
- Describe each field via a FieldDescriptor vector
- Pack itself into a raw byte buffer (used by HeaderBuffer to write)
- Unpack itself from a raw byte buffer (used by HeaderBuffer to read)
- Display its contents for debugging

NOT responsible for:

- Opening or closing files (HeaderBuffer does that)
- Writing or reading from streams (HeaderBuffer does that)

Precondition

All string fields must not require binary-safe escaping (no embedded nulls).

Postcondition

An initialised DataFileHeader can be packed into a buffer and written by HeaderBuffer as the first record of a .li file.

3.6.2 Constructor & Destructor Documentation

3.6.2.1 DataFileHeader() [1/2]

```
DataFileHeader::DataFileHeader ()
```

Default constructor — initialises all fields to safe defaults.

Precondition

None.

Postcondition

fileStructureType == "ZIPCODE_V2", version == 2, recordCount == 0, fieldCount == 0, primaryKeyFieldIndex == -1.

3.6.2.2 DataFileHeader() [2/2]

```
DataFileHeader::DataFileHeader (
            const std::string & structType,
            int ver)
```

Parameterised constructor.

Parameters

structType	File structure type string.
ver	Version number.

Precondition

structType is non-empty; ver $>$ 0.

Postcondition

Supplied values set; other fields take safe defaults.

3.6.3 Member Function Documentation

3.6.3.1 addField()

```
void DataFileHeader::addField (
            const std::string & name,
            const std::string & typeSchema,
            bool isPrimaryKey = false)
```

Append a field descriptor and update fieldCount.

Parameters

name	Field name.
typeSchema	Type string.
isPrimaryKey	True if this field is the primary key.

Precondition

name and typeSchema are non-empty.

Postcondition

Field appended to fields; fieldCount incremented; primaryKeyFieldIndex updated if isPrimaryKey is true.

3.6.3.2 display()

```
void DataFileHeader::display () const
```

Print all header fields to std::cout in a formatted layout.

Precondition

None.

Postcondition

Contents printed to standard output; no state change.

3.6.3.3 getPackedSize()

```
size_t DataFileHeader::getPackedSize () const
```

Calculate the number of bytes needed to pack this header.

Precondition

None.

Postcondition

No state change.

Returns

Required buffer size in bytes.

3.6.3.4 pack()

```
size_t DataFileHeader::pack (
            char * buffer) const
```

Serialise this header into a raw byte buffer.

Parameters

buffer	Caller-allocated buffer of at least getPackedSize() bytes.

Precondition

buffer points to valid memory of at least getPackedSize() bytes.

Postcondition

Header data written to buffer in binary format.

Returns

Number of bytes written.

3.6.3.5 unpack()

```
size_t DataFileHeader::unpack (
            const char * buffer)
```

Deserialise this header from a raw byte buffer.

Parameters

buffer	Buffer containing a previously packed header.

Precondition

buffer contains valid packed header data (produced by pack()).

Postcondition

All fields populated from buffer; fields vector rebuilt.

Returns

Number of bytes consumed.

3.6.4 Member Data Documentation

3.6.4.1 fieldCount

```
int DataFileHeader::fieldCount
```

Number of fields per data record

3.6.4.2 fields

```
std::vector<FieldDescriptor> DataFileHeader::fields
```

One entry per data-record field

3.6.4.3 fileStructureType

```
std::string DataFileHeader::fileStructureType
```

e.g. "ZIPCODE_V2"

3.6.4.4 headerSize

```
int DataFileHeader::headerSize
```

Size of the packed header in bytes

3.6.4.5 primaryKeyFieldIndex

```
int DataFileHeader::primaryKeyFieldIndex
```

Index into fields[] of the PK (-1 if none)

3.6.4.6 primaryKeyIndexFileName

```
std::string DataFileHeader::primaryKeyIndexFileName
```

Name of the associated .idx file

3.6.4.7 recordCount

```
int DataFileHeader::recordCount
```

Number of data records in the file

3.6.4.8 recordSizeIntegerBytes

```
int DataFileHeader::recordSizeIntegerBytes
```

Bytes used for the record-length prefix (2)

3.6.4.9 sizeFormatType

```
std::string DataFileHeader::sizeFormatType
```

e.g. "binary"

3.6.4.10 sizeInclusionFlag

```
bool DataFileHeader::sizeInclusionFlag
```

True if size prefix includes itself

3.6.4.11 sizeOfSizes

```
int DataFileHeader::sizeOfSizes
```

Bytes per size field

3.6.4.12 version

```
int DataFileHeader::version
```

Schema version number

The documentation for this class was generated from the following files:

- DataFileHeader.h
- DataFileHeader.cpp

3.7 FieldDescriptor Struct Reference

Describes a single field in each data record.

```
#include <DataFileHeader.h>
```

Public Member Functions

- FieldDescriptor ()

  Default constructor — isPrimaryKey defaults to false.
- FieldDescriptor (const std::string &n, const std::string &t, bool pk)

  Parameterised constructor.

Public Attributes

- std::string name
- std::string typeSchema
- bool isPrimaryKey

3.7.1 Detailed Description

Describes a single field in each data record.

Precondition

name and typeSchema must be non-empty strings.

Postcondition

Instances can be stored in a DataFileHeader's fields vector.

3.7.2 Constructor & Destructor Documentation

3.7.2.1 FieldDescriptor() [1/2]

```
FieldDescriptor::FieldDescriptor () [inline]
```

Default constructor — isPrimaryKey defaults to false.

Precondition

None.

Postcondition

All fields initialised to empty/false.

3.7.2.2 FieldDescriptor() [2/2]

```
FieldDescriptor::FieldDescriptor (
            const std::string & n,
            const std::string & t,
            bool pk) [inline]
```

Parameterised constructor.

Parameters

n	Field name.

t	Type schema string.
pk	True if this is the primary key field.

Precondition

None.

Postcondition

All members set to supplied values.

3.7.3 Member Data Documentation

3.7.3.1 isPrimaryKey

```
bool FieldDescriptor::isPrimaryKey
```

True if this field is the primary key

3.7.3.2 name

```
std::string FieldDescriptor::name
```

Field name (e.g. "zipCode", "placeName")

3.7.3.3 typeSchema

```
std::string FieldDescriptor::typeSchema
```

Field type (e.g. "int", "string", "double")

The documentation for this struct was generated from the following file:

- DataFileHeader.h

3.8 HeaderBuffer Class Reference

Writes and reads a DataFileHeader as the first record of a .li file.

```
#include <HeaderBuffer.h>
```

Public Member Functions

- HeaderBuffer ()=default

  Default constructor.
- ~HeaderBuffer ()=default

  Default destructor.
- bool writeHeader (std::ofstream &out, DataFileHeader &header)

  Pack a DataFileHeader and write it as the first length-indicated record of a binary output stream.
- bool readHeader (std::ifstream &in, DataFileHeader &header)

  Read the first length-indicated record and unpack it into a DataFileHeader.

3.8.1 Detailed Description

Writes and reads a DataFileHeader as the first record of a .li file.

Responsibilities:

- writeHeader() – packs the header and writes it with a 2-byte length prefix to byte 0 of an open output stream
- readHeader() – reads the 2-byte length, reads the payload, and calls DataFileHeader::unpack() to restore all fields

NOT responsible for:

- Opening or closing files (caller does that)
- Storing the DataFileHeader (caller owns it)

Precondition

Streams passed to read/write methods must already be open in binary mode and positioned at byte 0.

Postcondition

Stream position advances past the header record after each call.

3.8.2 Constructor & Destructor Documentation

3.8.2.1 HeaderBuffer()

```
HeaderBuffer::HeaderBuffer () [default]
```

Default constructor.

Precondition

None.

Postcondition

A HeaderBuffer instance is ready to use; no file is associated.

3.8.2.2 ~HeaderBuffer()

```
HeaderBuffer::~HeaderBuffer () [default]
```

Default destructor.

Precondition

None.

Postcondition

No resources to release.

3.8.3 Member Function Documentation

3.8.3.1 readHeader()

```
bool HeaderBuffer::readHeader (
            std::ifstream & in,
            DataFileHeader & header)
```

Read the first length-indicated record and unpack it into a DataFileHeader.

Read first LI record and unpack it into a DataFileHeader.

Parameters

in	Open std::ifstream in binary mode, positioned at byte 0.
header	DataFileHeader reference to populate.

Returns

true on success; false on I/O or unpack error.

Precondition

in is open in std::ios::binary mode at position 0.

Postcondition

header is fully populated; stream advances past header record.

Parameters

in	Open binary ifstream at byte 0.
header	Reference to populate.

Returns

true on success; false on any error.

Precondition

in is open in binary mode at position 0.

Postcondition

header is fully populated; stream advances past header record.

3.8.3.2 writeHeader()

```
bool HeaderBuffer::writeHeader (
            std::ofstream & out,
            DataFileHeader & header)
```

Pack a DataFileHeader and write it as the first length-indicated record of a binary output stream.

Serialise header and write it as the first LI record.

Parameters

out	Open std::ofstream in binary mode, positioned at byte 0.
header	The DataFileHeader to serialise and write.

Returns

true on success; false on I/O error.

Precondition

out is open in std::ios::binary mode at position 0.

Postcondition

Header record written; stream advances past it.

Parameters

out	Open binary ofstream at byte 0.
header	Header to write.

Returns

true on success; false on any error.

Precondition

out is open in binary mode at position 0.

Postcondition

Header occupies first (2 + payloadLen) bytes; stream advances.

The documentation for this class was generated from the following files:

- HeaderBuffer.h
- HeaderBuffer.cpp

3.9 LIBuffer Class Reference

Buffered reader/writer for length-indicated ZIP code data files.

```
#include <LIBuffer.h>
```

Public Member Functions

- LIBuffer (const std::string &filename)

  Construct an LIBuffer associated with a filename.
- ~LIBuffer ()

  Destructor — closes any open streams.
- bool openForRead ()

  Open the file for sequential reading.
- bool openForWrite ()

  Open (or create) the file for sequential writing. Truncates any existing content.
- void close ()

  Close any open read or write streams.
- bool readHeader (DataFileHeader &header)

  Read and unpack the header record from the file.
- bool getNextRecord (ZipCodeRecord &record)

  Read and unpack the next data record.
- bool readRecordAt (std::streamoff offset, ZipCodeRecord &record)

  Seek to a byte offset and read exactly one data record.
- bool writeHeader (DataFileHeader &header)

  Write the header record as the first LI record in the file.
- bool writeRecord (const ZipCodeRecord &record)

  Serialise and append one ZipCodeRecord to the file.
- std::streamoff getCurrentOffset () const

  Return the byte offset of the current read position.
- long getRecordCount () const

  Return the number of data records read or written so far.
- std::string getFilename () const

  Return the filename associated with this buffer.

3.9.1 Detailed Description

Buffered reader/writer for length-indicated ZIP code data files.

Write workflow:

1. Construct with filename
2. openForWrite()
3. writeHeader()
4. writeRecord() for each data record
5. close()

Read workflow:

1. Construct with filename
2. openForRead()
3. readHeader()
4. getNextRecord() until false
5. close()

Indexed read workflow:

1. openForRead()
2. readHeader()
3. readRecordAt(offset) for any record directly

Precondition

The file directory must be writable for write operations.

Postcondition

After close() the file is a valid LI ZIP code file.

3.9.2 Constructor & Destructor Documentation

3.9.2.1 LIBuffer()

```
LIBuffer::LIBuffer (
            const std::string & filename)  [explicit]
```

Construct an LIBuffer associated with a filename.

Construct an LIBuffer for the given filename.

Parameters

filename	Path to the .li binary data file.

Precondition

None.

Postcondition

Object initialised; no file opened yet.

Parameters

filename	Path to the .li data file.

Precondition

None.

Postcondition

Object ready; no file opened.

3.9.2.2 ∼LIBuffer()

```
LIBuffer::~LIBuffer ()
```

Destructor — closes any open streams.

Destructor — delegates to close().

Precondition

None.

Postcondition

All open streams are flushed and closed.

Precondition

None.

Postcondition

All open streams closed.

3.9.3 Member Function Documentation

3.9.3.1 close()

```
void LIBuffer::close ()
```

Close any open read or write streams.

Close any open streams.

Precondition

None.

Postcondition

Both streams closed; internal flags reset.

Precondition

None.

Postcondition

Both streams closed; flags reset.

3.9.3.2 getCurrentOffset()

```
std::streamoff LIBuffer::getCurrentOffset () const
```

Return the byte offset of the current read position.

Return the current byte offset of the read stream.

Precondition

openForRead() called.

Postcondition

No state change.

Returns

Current byte offset; -1 if not open.

Precondition

openForRead() called.

Postcondition

No state change.

Returns

Byte offset; -1 if not open.

3.9.3.3 getFilename()

```
std::string LIBuffer::getFilename () const
```

Return the filename associated with this buffer.

Return the filename.

Precondition

None.

Postcondition

No state change.

Returns

The filename string.

Precondition

None.

Postcondition

No state change.

Returns

filename string.

3.9.3.4 getNextRecord()

```
bool LIBuffer::getNextRecord (
            ZipCodeRecord & record)
```

Read and unpack the next data record.

Parameters

record	Output: populated on success.

Returns

true on success; false at EOF or on error.

Precondition

openForRead() and readHeader() called.

Postcondition

record populated; recordCount incremented; stream advances.

3.9.3.5 getRecordCount()

```
long LIBuffer::getRecordCount () const
```

Return the number of data records read or written so far.

Return the cumulative data record count.

Precondition

None.

Postcondition

No state change.

Returns

Cumulative data record count.

Precondition

None.

Postcondition

No state change.

Returns

recordCount.

3.9.3.6 openForRead()

```
bool LIBuffer::openForRead ()
```

Open the file for sequential reading.

Returns

true on success; false if file cannot be opened.

Precondition

File must exist and be a valid LI ZIP code file.

Postcondition

Stream positioned at byte 0; ready for readHeader().

Returns

true on success; false if file cannot be opened.

Precondition

None.

Postcondition

inStream open in binary mode at byte 0; readOpen == true.

3.9.3.7 openForWrite()

```
bool LIBuffer::openForWrite ()
```

Open (or create) the file for sequential writing. Truncates any existing content.

Open (or create) the file for sequential writing, truncating content.

Returns

true on success; false if file cannot be opened.

Precondition

Directory must be writable.

Postcondition

Stream positioned at byte 0; ready for writeHeader().

Returns

true on success; false if file cannot be opened.

Precondition

None.

Postcondition

outStream open in binary mode; writeOpen == true.

3.9.3.8 readHeader()

```
bool LIBuffer::readHeader (
            DataFileHeader & header)
```

Read and unpack the header record from the file.

Read and unpack the header record.

Parameters

header	Output: populated with file metadata on success.

Returns

true on success; false on I/O or unpack error.

Precondition

openForRead() returned true; stream at byte 0.

Postcondition

header populated; stream advances past header record.

Parameters

header	Output: populated with file metadata.

Returns

true on success; false on error.

Precondition

openForRead() returned true; stream at byte 0.

Postcondition

header populated; stream advances past header record.

3.9.3.9 readRecordAt()

```
bool LIBuffer::readRecordAt (
            std::streamoff offset,
            ZipCodeRecord & record)
```

Seek to a byte offset and read exactly one data record.

Parameters

offset	Byte offset of the record's 2-byte length prefix.
record	Output: populated on success.

Returns

true on success; false on seek/read/unpack error.

Precondition

openForRead() returned true.

Postcondition

record populated; stream positioned after the record.

Note

Used by IndexLoader for O(1) direct access.

Parameters

offset	Byte offset of the record's 2-byte length prefix.
record	Output: populated on success.

Returns

true on success; false on seek/read/unpack error.

Precondition

openForRead() returned true.

Postcondition

record populated; stream positioned after the record.

3.9.3.10 writeHeader()

```
bool LIBuffer::writeHeader (
            DataFileHeader & header)
```

Write the header record as the first LI record in the file.

Write the header record as the first LI record.

Parameters

header	The metadata to write.

Returns

true on success; false on I/O error.

Precondition

openForWrite() returned true; must be the first write call.

Postcondition

Header occupies first bytes of file; stream advances.

Parameters

header	Metadata to write.

Returns

true on success; false on I/O error.

Precondition

openForWrite() returned true; must be first write call.

Postcondition

Header written; stream advances.

3.9.3.11 writeRecord()

```
bool LIBuffer::writeRecord (
            const ZipCodeRecord & record)
```

Serialise and append one ZipCodeRecord to the file.

Serialise and append one ZipCodeRecord.

Parameters

record	The ZIP code data to write.

Returns

true on success; false on I/O error.

Precondition

openForWrite() and writeHeader() called.

Postcondition

One LI data record appended; recordCount incremented.

Parameters

record	ZIP code data to write.

Returns

true on success; false on I/O error.

Precondition

openForWrite() and writeHeader() called.

Postcondition

One LI data record appended; recordCount incremented.

The documentation for this class was generated from the following files:

- LIBuffer.h
- LIBuffer.cpp

3.10 RecordBuffer Class Reference

Converts ZipCodeRecord objects to/from raw bytes for block storage.

```
#include <RecordBuffer.h>
```

Public Member Functions

- **RecordBuffer** ()=default

  Default constructor.
- ~**RecordBuffer** ()=default

  Default destructor.
- std::string buildPayload (const ZipCodeRecord &record) const

  Build the ASCII payload for a record.
- size_t getPackedSize (const ZipCodeRecord &record) const

  Return the number of bytes needed to store this record, including the 2-byte length prefix.
- size_t pack (const ZipCodeRecord &record, char *buffer) const

  Pack a record into a raw byte buffer.
- size_t unpack (const char *buffer, ZipCodeRecord &record) const

  Unpack a record from a raw byte buffer.
- uint16_t readLength (const char *buffer) const

  Read only the payload length from a packed record.
- void writeLength (uint16_t length, char *buffer) const

  Write a 2-byte big-endian length prefix.

3.10.1 Detailed Description

Converts ZipCodeRecord objects to/from raw bytes for block storage.

Responsibilities:

- compute packed size of one record
- pack one record into a caller-provided buffer
- unpack one record from a raw buffer
- encode/decode the 2-byte big-endian length prefix

NOT responsible for:

- file I/O
- whole-block management
- indexing

3.10.2 Member Function Documentation

3.10.2.1 buildPayload()

```
std::string RecordBuffer::buildPayload (
            const ZipCodeRecord & record) const
```

Build the ASCII payload for a record.

Parameters

record	Input ZIP code record.

Returns

Comma-separated ASCII payload string.

3.10.2.2 getPackedSize()

```
size_t RecordBuffer::getPackedSize (
            const ZipCodeRecord & record) const
```

Return the number of bytes needed to store this record, including the 2-byte length prefix.

Parameters

record	Input ZIP code record.

Returns

Total packed byte count.

3.10.2.3 pack()

```
size_t RecordBuffer::pack (
            const ZipCodeRecord & record,
            char * buffer) const
```

Pack a record into a raw byte buffer.

Parameters

record	Input ZIP code record.
buffer	Caller-allocated output buffer.

Returns

Number of bytes written.

Precondition

buffer must have at least getPackedSize(record) bytes.

Postcondition

buffer contains [2-byte len][ASCII payload].

3.10.2.4 readLength()

```
uint16_t RecordBuffer::readLength (
            const char * buffer) const
```

Read only the payload length from a packed record.

Parameters

buffer	Input raw bytes beginning at a length prefix.

Returns

Payload length in bytes.

3.10.2.5 unpack()

```
size_t RecordBuffer::unpack (
            const char * buffer,
            ZipCodeRecord & record) const
```

Unpack a record from a raw byte buffer.

Parameters

buffer	Input raw bytes beginning at a length prefix.
record	Output record.

Returns

Number of bytes consumed, or 0 on error.

Precondition

buffer points to a valid packed record.

Postcondition

record populated if return value > 0.

3.10.2.6 writeLength()

```
void RecordBuffer::writeLength (
            uint16_t length,
            char * buffer) const
```

Write a 2-byte big-endian length prefix.

Parameters

length	Payload length.
buffer	Output buffer with space for at least 2 bytes.

The documentation for this class was generated from the following files:

- RecordBuffer.h
- RecordBuffer.cpp

3.11 ZipCodeRecord Struct Reference

Structure to hold information for a single ZIP code This structure contains all fields from the US Postal Code database:

```
#include <ZipCodeRecord.h>
```

Public Member Functions

- ZipCodeRecord ()

  Default constructor.
- ZipCodeRecord (int zip, const std::string &place, const std::string &st, const std::string &cnty, double lat, double lon)

  Parameterized constructor.
- size_t getPackedSize () const

  Calculate the packed size of this record.
- void display () const

  Display record information.
- void displayInline () const

  Display record in a single line.

Public Attributes

- int **zipCode**
- std::string **placeName**
- std::string **state**
- std::string **county**
- double **latitude**
- double **longitude**

3.11.1 Detailed Description

Structure to hold information for a single ZIP code This structure contains all fields from the US Postal Code database:

- ZIP code (5-digit postal code)
- Place name (city/town name)
- State (2-character state abbreviation)
- County name
- Geographic coordinates (latitude and longitude) Longitude values are negative for locations west of the Prime Meridian, and latitude values are positive for locations north of the Equator.

  Precondition

  All fields must be properly initialized before use

  Postcondition

  Instances of ZipCodeRecord can be used to store and retrieve ZIP code information

  Note

  This structure serves as the fundamental data unit for managing and processing ZIP code information in our application.

3.11.2 Constructor & Destructor Documentation

3.11.2.1 ZipCodeRecord() [1/2]

```
ZipCodeRecord::ZipCodeRecord () [inline]
```

Default constructor.

Precondition

None

Postcondition

All fields are initialized to default values

3.11.2.2 ZipCodeRecord() [2/2]

```
ZipCodeRecord::ZipCodeRecord (
            int zip,
            const std::string & place,
            const std::string & st,
            const std::string & cnty,
            double lat,
            double lon) [inline]
```

Parameterized constructor.

Precondition

None

Postcondition

All fields are initialized to the provided values

3.11.3 Member Function Documentation

3.11.3.1 display()

```
void ZipCodeRecord::display () const [inline]
```

Display record information.

Precondition

None

Postcondition

Record information is printed to the console in a readable format

Note

This method is useful for debugging and verifying the contents of a ZipCodeRecord instance.

3.11.3.2 displayInline()

```
void ZipCodeRecord::displayInline () const [inline]
```

Display record in a single line.

Precondition

None

Postcondition

Record information is printed to the console in a single line format

Note

This method is useful for displaying multiple records in a compact format, such as in a list or table.

3.11.3.3 getPackedSize()

```
size_t ZipCodeRecord::getPackedSize () const [inline]
```

Calculate the packed size of this record.

Precondition

None

Postcondition

Returns the size in bytes when this record is packed for length-indicated format

Returns

Size in bytes when packed for length-indicated format

The documentation for this struct was generated from the following file:

- ZipCodeRecord.h

Chapter 4

File Documentation

4.1 add_seqset.cpp File Reference

Adds ZIP code records to the GP 3.0 sequence-set file (supports insert without split and insert with split).

```
#include <iostream>
#include <iomanip>
#include <string>
#include <vector>
#include <fstream>
#include <sstream>
#include <algorithm>
#include "BlockedFileHeader.h"
#include "BlockBuffer.h"
#include "BlockFileBuffer.h"
#include "BlockIndex.h"
#include "ZipCodeRecord.h"
```

Functions

- bool loadAddRecords (const std::string &filename, std::vector< ZipCodeRecord > &records)

  Forward declarations.
- bool parseAddLine (const std::string &line, ZipCodeRecord &record)

  Parses a single line from the add records file into a ZipCodeRecord.
- bool rebuildBlockIndex (const std::string &seqFilename, const std::string &idxFilename)

  Rebuilds the block index for the sequence-set file.
- bool addOneRecord (const ZipCodeRecord &record)

  Adds a single ZIP code record to the sequence-set, handling splits if necessary.
- void printRecord (const ZipCodeRecord &record)

  Prints the details of a ZipCodeRecord to standard output.
- int main ()

  Main function that adds ZIP code records to the sequence-set file.

4.1.1 Detailed Description

Adds ZIP code records to the GP 3.0 sequence-set file (supports insert without split and insert with split).

Author

CSCI 331 Team 2

Date

04/01/2026

4.1.2 Function Documentation

4.1.2.1 addOneRecord()

```
bool addOneRecord (
            const ZipCodeRecord & record)
```

Adds a single ZIP code record to the sequence-set, handling splits if necessary.

Parameters

record	The ZipCodeRecord to add.

Returns

true if addition succeeds, false otherwise.

4.1.2.2 loadAddRecords()

```
bool loadAddRecords (
            const std::string & filename,
            std::vector< ZipCodeRecord > & records)
```

Forward declarations.

Loads add records from the input file into a vector of ZipCodeRecord.

Parameters

filename	The name of the input file containing records to add.
records	The vector to store the loaded records.

Returns

true if loading succeeds, false otherwise.

4.1.2.3 main()

```
int main ()
```

Main function that adds ZIP code records to the sequence-set file.

Returns

0 on success, 1 on failure.

4.1.2.4 parseAddLine()

```
bool parseAddLine (
            const std::string & line,
            ZipCodeRecord & record)
```

Parses a single line from the add records file into a ZipCodeRecord.

Parameters

line	The input line to parse.
record	The ZipCodeRecord to populate.

Returns

true if parsing succeeds, false otherwise.

4.1.2.5 printRecord()

```
void printRecord (
            const ZipCodeRecord & record)
```

Prints the details of a ZipCodeRecord to standard output.

Parameters

record	The ZipCodeRecord to print.

4.1.2.6 rebuildBlockIndex()

```
bool rebuildBlockIndex (
            const std::string & seqFilename,
            const std::string & idxFilename)
```

Rebuilds the block index for the sequence-set file.

Parameters

seqFilename	The sequence-set file name.
idxFilename	The index file name.

Returns

true if rebuild succeeds, false otherwise.

4.2 BlockBuffer.cpp File Reference

Implementation of BlockBuffer.

```
#include "BlockBuffer.h"
#include <algorithm>
#include <cstring>
#include <iostream>
#include <iomanip>
```

4.2.1 Detailed Description

Implementation of BlockBuffer.

Author

CSCI 331 Team 2

Date

04/01/2026

4.3 BlockBuffer.h File Reference

Represents one fixed-size block in the blocked sequence-set file.

```
#include <vector>
#include <string>
#include <cstddef>
#include <cstdint>
#include "ZipCodeRecord.h"
#include "RecordBuffer.h"
```

Classes

- class BlockBuffer

  In-memory representation of one physical block.

4.3.1 Detailed Description

Represents one fixed-size block in the blocked sequence-set file.

Author

CSCI 331 Team 2

Date

04/01/2026

A block stores:

- a block header
- zero or more variable-length ZIP code records

Active block layout: [int32 recordCount] [int32 prevRBN] [int32 nextRBN] [record bytes...] [unused space padded with blanks]

Avail block layout: [int32 recordCount == 0] [int32 nextAvailRBN] [remaining bytes padded with blanks]

For active blocks:

- recordCount $>$ 0
- records are stored sorted by ZIP code

For avail blocks:

- recordCount == 0
- prevRBN is not used
- nextRBN is interpreted as nextAvailRBN

4.4 BlockBuffer.h

Go to the documentation of this file.

```
00033
00034 #ifndef BLOCKBUFFER_H
00035 #define BLOCKBUFFER_H
00036
00037 #include <vector>
00038 #include <string>
00039 #include <cstddef>
00040 #include <cstdint>
00041 #include "ZipCodeRecord.h"
00042 #include "RecordBuffer.h"
00043
00061 class BlockBuffer
00062 {
00063 public:
00068     explicit BlockBuffer(int blockSize = 512);
00069
00073     void clearToEmptyActive();
00074
00079     void clearToAvail(int nextAvailRBN = -1);
00080
00085     bool isAvailBlock() const;
00086
00091     bool isActiveBlock() const;
00092
00097     int getBlockSize() const;
00098
00103     int getRecordCount() const;
00104
00109     void setPrevRBN(int rbn);
00110
00115     void setNextRBN(int rbn);
00116
00121     int getPrevRBN() const;
00122
00127     int getNextRBN() const;
00128
00133     size_t getUsedBytes() const;
00134
00139     size_t getFreeBytes() const;
00140
00146     bool canFit(const ZipCodeRecord& record) const;
00147
00153     bool insertRecordSorted(const ZipCodeRecord& record);
00154
00160     bool removeRecordByZip(int zipCode);
00161
00168     bool findRecord(int zipCode, ZipCodeRecord& record) const;
00169
00174     int getHighestKey() const;
00175
00180     const std::vector<ZipCodeRecord>& getRecords() const;
00181
00187     bool pack(char* buffer) const;
00188
00194     bool unpack(const char* buffer);
00195
00199     void display() const;
00200
00201 private:
00202     int blockSize;
00203     int recordCount;
00204     int prevRBN;
00205     int nextRBN;
00206     std::vector<ZipCodeRecord> records;
00207     RecordBuffer recordBuffer;
00208
00213     size_t getBlockHeaderSize() const;
00214
00218     void sortRecords();
00219 };
00220
00221 #endif // BLOCKBUFFER_H
```

4.5 BlockedFileHeader.cpp File Reference

Implementation of BlockedFileHeader.

```
#include "BlockedFileHeader.h"
```

4.5.1 Detailed Description

Implementation of BlockedFileHeader.

Author

CSCI 331 Team 2

Date

04/01/2026

4.6 BlockedFileHeader.h File Reference

Header structure for the GP 3.0 blocked sequence-set ZIP code file.

```
#include <string>
#include <vector>
#include <iostream>
#include <iomanip>
#include <cstring>
#include "DataFileHeader.h"
```

Classes

- class BlockedFileHeader

  Stores metadata for a blocked sequence-set file.

4.6.1 Detailed Description

Header structure for the GP 3.0 blocked sequence-set ZIP code file.

Author

CSCI 331 Team 2

Date

04/01/2026

This class stores file-level metadata for the blocked sequence-set file. It extends the ideas from the GP 2.0 header and adds blocked-file fields:

- block size
- minimum block capacity
- active list head
- avail list head
- block count
- stale flag
- index file metadata

The header is stored as the FIRST length-indicated record in the file, just like in Project 2.0.

4.7 BlockedFileHeader.h

Go to the documentation of this file.

```
00001
00021
00022 #ifndef BLOCKEDFILEHEADER_H
00023 #define BLOCKEDFILEHEADER_H
00024
00025 #include <string>
00026 #include <vector>
00027 #include <iostream>
00028 #include <iomanip>
00029 #include <cstring>
00030 #include "DataFileHeader.h"
00031
00047 class BlockedFileHeader
00048 {
00049 public:
00050     // -- General file metadata ---------------------------------------------
00051     std::string fileStructureType;
00052     int         version;
00053     int         headerSize;
00054
00055     // -- Size format metadata ----------------------------------------------
00056     int         recordSizeIntegerBytes;
00057     std::string sizeFormatType;
00058     int         sizeOfSizes;
00059     bool        sizeInclusionFlag;
00060
00061     // -- GP 3.0 blocked-file metadata --------------------------------------
00062     int         blockSize;
00063     double      minBlockCapacity;
00064     int         recordCount;
00065     int         blockCount;
00066     int         activeListHead;
00067     int         availListHead;
00068     bool        staleFlag;
00069
00070     // -- Index metadata ----------------------------------------------------
00071     std::string primaryKeyIndexFileName;
00072     std::string indexFileStructureType;
00073     int         primaryKeyFieldIndex;
00074
00075     // -- Field metadata ----------------------------------------------------
00076     int fieldCount;
00077     std::vector<FieldDescriptor> fields;
00078
00079     // -- Constructors ------------------------------------------------------
00080     BlockedFileHeader();
00081     BlockedFileHeader(const std::string& structType, int ver);
00082
00083     // -- Field management --------------------------------------------------
00084     void addField(const std::string& name,
00085                   const std::string& typeSchema,
00086                   bool isPrimaryKey = false);
00087
00088     // -- Serialization -----------------------------------------------------
00089     size_t getPackedSize() const;
00090     size_t pack(char* buffer) const;
00091     size_t unpack(const char* buffer);
00092
00093     // -- Utility -----------------------------------------------------------
00094     void display() const;
00095 };
00096
00097 #endif // BLOCKEDFILEHEADER_H
```

4.8 BlockFileBuffer.cpp File Reference

Implementation of BlockFileBuffer.

```
#include "BlockFileBuffer.h"
#include <iostream>
#include <vector>
#include <cstdint>
#include <cstring>
```

4.8.1 Detailed Description

Implementation of BlockFileBuffer.

Author

CSCI 331 Team 2

Date

04/01/2026

4.9 BlockFileBuffer.h File Reference

File I/O layer for blocked sequence-set files.

```
#include <fstream>
#include <string>
#include <streambuf>
#include "BlockedFileHeader.h"
#include "BlockBuffer.h"
```

Classes

- class BlockFileBuffer

  Provides physical file access for blocked sequence-set files.

4.9.1 Detailed Description

File I/O layer for blocked sequence-set files.

Author

CSCI 331 Team 2

Date

04/01/2026

This class manages the physical blocked file on disk.

File layout: [2-byte header length][header payload] [block 0 bytes] [block 1 bytes] [block 2 bytes] ...

Responsibilities:

- open/close the file
- write/read the blocked file header
- write/read one fixed-size block by RBN
- compute byte offsets for blocks

NOT responsible for:

- sequence-set logic
- block splitting/merging
- indexing

4.10 BlockFileBuffer.h

Go to the documentation of this file.

```
00001
00028
00029 #ifndef BLOCKFILEBUFFER_H
00030 #define BLOCKFILEBUFFER_H
00031
00032 #include <fstream>
00033 #include <string>
00034 #include <streambuf>
00035 #include "BlockedFileHeader.h"
00036 #include "BlockBuffer.h"
00037
00042 class BlockFileBuffer
00043 {
00044 public:
00049     explicit BlockFileBuffer(const std::string& filename);
00050
00054     ~BlockFileBuffer();
00055
00060     bool openForRead();
00061
00066     bool openForWrite();
00067
00072     bool openForUpdate();
00073
00077     void close();
00078
00087     bool writeHeader(BlockedFileHeader& header);
00088
00097     bool readHeader(BlockedFileHeader& header);
00098
00105     bool writeBlock(int rbn, const BlockBuffer& block);
00106
00113     bool readBlock(int rbn, BlockBuffer& block);
00114
00121     std::streamoff getBlockOffset(int rbn, int blockSize) const;
00122
00127     std::string getFilename() const;
00128
00133     bool isReadOpen() const;
00134
00139     bool isWriteOpen() const;
00140
00141 private:
00142     std::string filename;
00143     std::ifstream inStream;
00144     std::fstream ioStream;
00145     bool readOpen;
00146     bool writeOpen;
00147     std::streamoff headerEndPos;
00148
00155     bool readLength(std::istream& in, uint16_t& length);
00156
00163     bool writeLength(std::ostream& out, uint16_t length);
00164 };
00165
00166 #endif // BLOCKFILEBUFFER_H
```

4.11 BlockIndex.cpp File Reference

Implementation of BlockIndex.

```
#include "BlockIndex.h"
#include <fstream>
#include <iostream>
#include <algorithm>
```

4.11.1 Detailed Description

Implementation of BlockIndex.

Author

CSCI 331 Team 2

Date

04/01/2026

4.12 BlockIndex.h File Reference

Simple primary-key block index for GP 3.0.

```
#include <string>
#include <vector>
#include <cstddef>
#include <cstdint>
```

Classes

- struct BlockIndexEntry

  One simple block-index entry.
- class BlockIndex

  In-memory simple index for the blocked sequence-set file.

4.12.1 Detailed Description

Simple primary-key block index for GP 3.0.

Author

CSCI 331 Team 2

Date

04/01/2026

This index stores one entry per active block:

[highest key in block] -> [RBN]

The entries are sorted by highest key so binary search can be used.

4.13 BlockIndex.h

Go to the documentation of this file.

```
00001
00014
00015 #ifndef BLOCKINDEX_H
00016 #define BLOCKINDEX_H
00017
00018 #include <string>
00019 #include <vector>
00020 #include <cstddef>
00021 #include <cstdint>
00022
00027 struct BlockIndexEntry
00028 {
00029     int32_t maxKey;
00030     int32_t rbn;
00031
00032     BlockIndexEntry() : maxKey(0), rbn(-1) {}
00033     BlockIndexEntry(int key, int blockNum) : maxKey(key), rbn(blockNum) {}
00034
00035     bool operator<(const BlockIndexEntry& other) const
00036     {
00037         return maxKey < other.maxKey;
00038     }
00039 };
00040
00052 class BlockIndex
00053 {
00054 public:
00059     explicit BlockIndex(const std::string& indexFilename);
00060
00064     void clear();
00065
00071     void addEntry(int maxKey, int rbn);
00072
00076     void sortEntries();
00077
00082     bool writeIndex() const;
00083
00088     bool loadIndex();
00089
00100     bool findCandidateBlock(int zipCode, int& rbn) const;
00101
00106     size_t size() const;
00107
00112     std::string getFilename() const;
00113
00118     const std::vector<BlockIndexEntry>& getEntries() const;
00119
00123     void display() const;
00124
00125 private:
00126     std::string indexFilename;
00127     std::vector<BlockIndexEntry> entries;
00128 };
00129
00130 #endif // BLOCKINDEX_H
```

4.14 build_block_index.cpp File Reference

Builds the GP 3.0 simple block index from the sequence-set file.

```
#include <iostream>
#include <string>
#include "BlockedFileHeader.h"
#include "BlockBuffer.h"
#include "BlockFileBuffer.h"
#include "BlockIndex.h"
```

Functions

- bool **buildBlockIndex** (const std::string &seqFilename, const std::string &idxFilename)
- int **main** ()

4.14.1 Detailed Description

Builds the GP 3.0 simple block index from the sequence-set file.

Author

CSCI 331 Team 2

Date

04/01/2026

4.15 build_seqset.cpp File Reference

Builds the GP 3.0 blocked sequence-set file from a GP 2.0 LI file.

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <cstdint>
#include "ZipCodeRecord.h"
#include "RecordBuffer.h"
#include "BlockedFileHeader.h"
#include "BlockBuffer.h"
#include "BlockFileBuffer.h"
```

Functions

- BlockedFileHeader **buildBlockedHeader** ()
- bool **buildSequenceSet** (const std::string &inputLI, const std::string &outputSeq, int blockSize)
- bool **readNextLIRecord** (std::ifstream &in, RecordBuffer &recordBuffer, ZipCodeRecord &record)
- bool **skipLIHeader** (std::ifstream &in)
- uint16_t **readBigEndianLength** (std::istream &in)
- int **main** ()

4.15.1 Detailed Description

Builds the GP 3.0 blocked sequence-set file from a GP 2.0 LI file.

Author

CSCI 331 Team 2

Date

04/01/2026

4.16 DataFileHeader.cpp File Reference

Implementation of DataFileHeader.

```
#include "DataFileHeader.h"
```

4.16.1 Detailed Description

Implementation of DataFileHeader.

Author

CSCI 331 Team 2

Date

04/01/2026

4.17 DataFileHeader.h File Reference

Data structure for the header record of a length-indicated ZIP code file.

```
#include <string>
#include <vector>
#include <iostream>
#include <iomanip>
#include <cstring>
```

Classes

- struct FieldDescriptor

  Describes a single field in each data record.
- class DataFileHeader

  Holds metadata about a length-indicated ZIP code data file.

4.17.1 Detailed Description

Data structure for the header record of a length-indicated ZIP code file.

Author

CSCI 331 Team 2

Date

03/13/2026

This struct holds all metadata about a .li data file. It knows how to pack/unpack itself into/from a raw byte buffer, but has NO file I/O — that responsibility belongs to HeaderBuffer.

The header is always stored as the FIRST length-indicated record inside the .li file (byte 0), using the same 2-byte length prefix format as every other record.

4.18 DataFileHeader.h

Go to the documentation of this file.

```
00001
00015
00016 #ifndef DATAFILEHEADER_H
00017 #define DATAFILEHEADER_H
00018
00019 #include <string>
00020 #include <vector>
00021 #include <iostream>
00022 #include <iomanip>
00023 #include <cstring>
00024
00031 struct FieldDescriptor
00032 {
00033     std::string name;
00034     std::string typeSchema;
00035     bool        isPrimaryKey;
00036
00042     FieldDescriptor() : isPrimaryKey(false) {}
00043
00052     FieldDescriptor(const std::string& n, const std::string& t, bool pk)
00053         : name(n), typeSchema(t), isPrimaryKey(pk) {}
00054 };
00055
00056
00076 class DataFileHeader
00077 {
00078 public:
00079     // -- Metadata fields -----------------------------------------------------
00080
00081     std::string fileStructureType;
00082     int         version;
00083     int         headerSize;
00084     int         recordSizeIntegerBytes;
00085     std::string sizeFormatType;
00086     int         sizeOfSizes;
00087     bool        sizeInclusionFlag;
00088     std::string primaryKeyIndexFileName;
00089     int         recordCount;
00090     int         fieldCount;
00091     int         primaryKeyFieldIndex;
00092
00093     std::vector<FieldDescriptor> fields;
00094
00095     // -- Constructors --------------------------------------------------------
00096
00103     DataFileHeader();
00104
00112     DataFileHeader(const std::string& structType, int ver);
00113
00114     // -- Field management ----------------------------------------------------
00115
00125     void addField(const std::string& name,
00126                   const std::string& typeSchema,
00127                   bool isPrimaryKey = false);
00128
00129     // -- Serialisation -------------------------------------------------------
00130
00137     size_t getPackedSize() const;
00138
00146     size_t pack(char* buffer) const;
00147
00155     size_t unpack(const char* buffer);
00156
00157     // -- Display -------------------------------------------------------------
00158
00164     void display() const;
00165 };
00166
00167 #endif // DATAFILEHEADER_H
```

4.19 delete_seqset.cpp File Reference

Deletes ZIP code records from the GP 3.0 sequence-set file (supports simple delete, redistribution, and merge).

```
#include <iostream>
#include <iomanip>
```

```
#include <string>
#include <vector>
#include <fstream>
#include <sstream>
#include <cctype>
#include <cmath>
#include "BlockedFileHeader.h"
#include "BlockBuffer.h"
#include "BlockFileBuffer.h"
#include "BlockIndex.h"
#include "ZipCodeRecord.h"
```

Functions

- bool **loadDeleteZips** (const std::string &filename, std::vector< int > &zips)
- bool **parseDeleteLine** (const std::string &line, int &zipCode)
- bool **rebuildBlockIndex** (const std::string &seqFilename, const std::string &idxFilename)
- bool **deleteOneZip** (int zipCode)
- size_t **computeMaxRecordsPerBlock** (int blockSize)
- size_t **computeMinRecordsRequired** (int blockSize, double minCapacity)
- bool **redistributeFromLeft** (BlockFileBuffer &file, const BlockedFileHeader &header, int leftRBN, BlockBuffer &left, int targetRBN, BlockBuffer &target)
- bool **redistributeFromRight** (BlockFileBuffer &file, const BlockedFileHeader &header, int rightRBN, BlockBuffer &right, int targetRBN, BlockBuffer &target)
- bool **mergeWithLeft** (BlockFileBuffer &file, BlockedFileHeader &header, int leftRBN, BlockBuffer &left, int targetRBN, BlockBuffer &target)
- bool **mergeWithRight** (BlockFileBuffer &file, BlockedFileHeader &header, int targetRBN, BlockBuffer &target, int rightRBN, BlockBuffer &right)
- int **main** ()

4.19.1 Detailed Description

Deletes ZIP code records from the GP 3.0 sequence-set file (supports simple delete, redistribution, and merge).

Author

CSCI 331 Team 2

Date

04/01/2026

4.20 dump_seqset.cpp File Reference

Dumps a GP 3.0 blocked sequence-set file.

```
#include <iostream>
#include <string>
#include "BlockedFileHeader.h"
#include "BlockBuffer.h"
#include "BlockFileBuffer.h"
```

Functions

- void **dumpPhysical** (BlockFileBuffer &file, const BlockedFileHeader &header)
- void **dumpLogical** (BlockFileBuffer &file, const BlockedFileHeader &header)
- int **main** ()

4.20.1 Detailed Description

Dumps a GP 3.0 blocked sequence-set file.

4.21 HeaderBuffer.cpp File Reference

Implementation of HeaderBuffer class.

```
#include "HeaderBuffer.h"
#include <iostream>
#include <cstdint>
```

4.21.1 Detailed Description

Implementation of HeaderBuffer class.

Author

CSCI 331 Team 2

Date

03/13/2026

All records in the .li file use a 2-byte big-endian uint16_t length prefix. The header record is no different — it is simply the first one in the file.

Write sequence:

1. Call DataFileHeader::getPackedSize() to find payload length
2. Allocate a buffer and call DataFileHeader::pack()
3. Write 2-byte big-endian length prefix
4. Write payload bytes

Read sequence:

1. Read 2 bytes -> decode big-endian length
2. Allocate buffer of that size, read payload bytes
3. Call DataFileHeader::unpack()

4.22 HeaderBuffer.h File Reference

Reads and writes the DataFileHeader as the first length-indicated record of a .li binary data file.

```
#include <fstream>
#include <string>
#include "DataFileHeader.h"
```

Classes

- class HeaderBuffer

  Writes and reads a DataFileHeader as the first record of a .li file.

4.22.1 Detailed Description

Reads and writes the DataFileHeader as the first length-indicated record of a .li binary data file.

Author

CSCI 331 Team 2

Date

03/13/2026

HeaderBuffer is responsible for ALL file I/O involving the header. It uses DataFileHeader::pack() and DataFileHeader::unpack() for serialisation, and writes/reads using a 2-byte big-endian length prefix — exactly the same format as every other record in the file.

Write layout at byte 0 of the .li file: [uint8_t hi][uint8_t lo] <- 2-byte big-endian payload length [payload bytes ...] <- DataFileHeader::pack() output

4.23 HeaderBuffer.h

Go to the documentation of this file.

```
00001
00017
00018 #ifndef HEADERBUFFER_H
00019 #define HEADERBUFFER_H
00020
00021 #include <fstream>
00022 #include <string>
00023 #include "DataFileHeader.h"
00024
00043 class HeaderBuffer
00044 {
00045 public:
00051     HeaderBuffer() = default;
00052
00058     ~HeaderBuffer() = default;
00059
00071     bool writeHeader(std::ofstream& out, DataFileHeader& header);
00072
00084     bool readHeader(std::ifstream& in, DataFileHeader& header);
00085 };
00086
00087 #endif // HEADERBUFFER_H
```

4.24 LIBuffer.cpp File Reference

Implementation of LIBuffer class.

```
#include "LIBuffer.h"
#include <iostream>
#include <sstream>
#include <iomanip>
#include <vector>
#include <string>
#include <cstring>
#include <cstdint>
```

4.24.1 Detailed Description

Implementation of LIBuffer class.

Author

CSCI 331 Team 2

Date

03/13/2026

Every record uses a 2-byte big-endian uint16_t length prefix. Data records are packed as raw binary (ints, doubles, c-strings). The header record is delegated to HeaderBuffer.

4.25 LIBuffer.h File Reference

Reads and writes length-indicated ZIP code data files.

```
#include <fstream>
#include <string>
#include <cstdint>
#include "ZipCodeRecord.h"
#include "DataFileHeader.h"
#include "HeaderBuffer.h"
```

Classes

- class LIBuffer

  Buffered reader/writer for length-indicated ZIP code data files.

4.25.1 Detailed Description

Reads and writes length-indicated ZIP code data files.

Author

CSCI 331 Team 2

Date

03/13/2026

A length-indicated (LI) file stores every record — header and data — as a 2-byte big-endian uint16_t length prefix followed by the packed payload bytes.

File layout: [2-byte len][header payload] <- HeaderBuffer writes/reads this [2-byte len][record payload] <- data record 1 [2-byte len][record payload] <- data record 2 ...

Data record payload (packed binary): [4-byte int zipCode] [c-string placeName\0] [c-string state\0] [c-string county\0] [8-byte double latitude] [8-byte double longitude]

Note

LIBuffer owns the file streams. HeaderBuffer and DataFileHeader are used as helpers.

4.26 LIBuffer.h

Go to the documentation of this file.

```
00001
00028
00029 #ifndef LIBUFFER_H
00030 #define LIBUFFER_H
00031
00032 #include <fstream>
00033 #include <string>
00034 #include <cstdint>
00035 #include "ZipCodeRecord.h"
00036 #include "DataFileHeader.h"
00037 #include "HeaderBuffer.h"
00038
00065 class LIBuffer
00066 {
00067 public:
00068     // -- Construction / Destruction ----------------------------------------
00069
00076     explicit LIBuffer(const std::string& filename);
00077
00083     ~LIBuffer();
00084
00085     // -- File lifecycle ----------------------------------------------------
00086
00093     bool openForRead();
00094
00102     bool openForWrite();
00103
00109     void close();
00110
00111     // -- Read interface ----------------------------------------------------
00112
00120     bool readHeader(DataFileHeader& header);
00121
00129     bool getNextRecord(ZipCodeRecord& record);
00130
00140     bool readRecordAt(std::streamoff offset, ZipCodeRecord& record);
00141
00142     // -- Write interface ---------------------------------------------------
```

```
    bool writeHeader(DataFileHeader& header);

    bool writeRecord(const ZipCodeRecord& record);

    // -- Accessors -------------------------------------------------------

    std::streamoff getCurrentOffset() const;

    long getRecordCount() const;

    std::string getFilename() const;

private:
    // -- Private helpers -------------------------------------------------

    size_t packRecord(const ZipCodeRecord& record, char* buffer) const;

    size_t unpackRecord(const char* buffer, ZipCodeRecord& record, size_t len) const;

    size_t getRecordPackedSize(const ZipCodeRecord& record) const;

    bool readLength(uint16_t& length);

    bool writeLength(uint16_t length);

    // -- Member variables ------------------------------------------------

    std::string   filename;
    std::ifstream inStream;
    std::ofstream outStream;
    bool          readOpen;
    bool          writeOpen;
    long          recordCount;
    HeaderBuffer  headerBuf;
};

#endif // LIBUFFER_H
```

4.27 RecordBuffer.cpp File Reference

Implementation of RecordBuffer.

```
#include "RecordBuffer.h"
#include <sstream>
#include <iomanip>
#include <cstring>
#include <iostream>
```

4.27.1 Detailed Description

Implementation of RecordBuffer.

Author

CSCI 331 Team 2

Date

04/01/2026

4.28 RecordBuffer.h File Reference

Packs and unpacks one ZIP code record for blocked sequence-set storage.

```
#include <string>
#include <vector>
#include <cstddef>
#include <cstdint>
#include "ZipCodeRecord.h"
```

Classes

- class RecordBuffer

  Converts ZipCodeRecord objects to/from raw bytes for block storage.

4.28.1 Detailed Description

Packs and unpacks one ZIP code record for blocked sequence-set storage.

Author

CSCI 331 Team 2

Date

04/01/2026

This class is responsible for converting a ZipCodeRecord to and from a raw byte representation stored inside a block.

GP 3.0 still expects the record fields to remain comma-separated in logical form, so this buffer stores each record as:

[2-byte big-endian payload length] [ASCII payload: ZZZZZ,place,state,county,lat,lon]

This allows records to remain variable-length while still being easy to pack into a fixed-size block.

4.29 RecordBuffer.h

Go to the documentation of this file.

```
00001
00020
00021 #ifndef RECORDBUFFER_H
00022 #define RECORDBUFFER_H
00023
00024 #include <string>
00025 #include <vector>
00026 #include <cstddef>
00027 #include <cstdint>
00028 #include "ZipCodeRecord.h"
00029
00046 class RecordBuffer
00047 {
00048 public:
00052     RecordBuffer() = default;
00053
00057     ~RecordBuffer() = default;
00058
00064     std::string buildPayload(const ZipCodeRecord& record) const;
00065
00072     size_t getPackedSize(const ZipCodeRecord& record) const;
00073
00083     size_t pack(const ZipCodeRecord& record, char* buffer) const;
00084
00094     size_t unpack(const char* buffer, ZipCodeRecord& record) const;
00095
00101     uint16_t readLength(const char* buffer) const;
00102
00108     void writeLength(uint16_t length, char* buffer) const;
00109
00110 private:
00117     bool parsePayload(const std::string& payload, ZipCodeRecord& record) const;
00118 };
00119
00120 #endif // RECORDBUFFER_H
```

4.30 search_seqset.cpp File Reference

Indexed ZIP code search for the GP 3.0 sequence-set file.

```
#include <iostream>
#include <iomanip>
#include <string>
#include <vector>
#include <cctype>
#include "BlockedFileHeader.h"
#include "BlockBuffer.h"
#include "BlockFileBuffer.h"
#include "BlockIndex.h"
#include "ZipCodeRecord.h"
```

Functions

- std::vector< int > **parseZipArgs** (int argc, char *argv[])
- bool **searchOneZip** (const std::string &seqFilename, const BlockIndex &index, int zipCode)
- int **main** (int argc, char *argv[])

4.30.1 Detailed Description

Indexed ZIP code search for the GP 3.0 sequence-set file.

Author

CSCI 331 Team 2

Date

04/01/2026

4.31 ZipCodeBuffer.cpp File Reference

Implementation of the ZipCodeBuffer class.

```
#include "ZipCodeBuffer.h"
#include <sstream>
#include <iostream>
#include <algorithm>
#include <cctype>
```

4.31.1 Detailed Description

Implementation of the ZipCodeBuffer class.

Author

CSCI 331 Team 2

Date

03/13/2026

Supports column re-ordering by reading the header row on first open and mapping column names to field indices. Data rows are then parsed using those indices regardless of column order.

Supported header column names (case-insensitive): zip, zip code, zipcode -> zipCode place, place name -> placeName state -> state county -> county lat, latitude -> latitude long, longitude -> longitude

4.32 ZipCodeRecord.h File Reference

Data structure for US Postal Code records.

```
#include <string>
#include <iostream>
#include <iomanip>
```

Classes

- struct ZipCodeRecord

  Structure to hold information for a single ZIP code This structure contains all fields from the US Postal Code database:

4.32.1 Detailed Description

Data structure for US Postal Code records.

Author

CSCI 331 Team 2

Date

03/13/2026

Version

2.0

4.33 ZipCodeRecord.h

Go to the documentation of this file.

```
00001
00008
00009 #ifndef ZIPCODERECORD_H
00010 #define ZIPCODERECORD_H
00011
00012 #include <string>
00013 #include <iostream>
00014 #include <iomanip>
00015
00030 struct ZipCodeRecord
00031 {
00032     int zipCode;
00033     std::string placeName;
00034     std::string state;
00035     std::string county;
00036     double latitude;
00037     double longitude;
00038
00044     ZipCodeRecord() : zipCode(0), latitude(0.0), longitude(0.0) {}
00045
00051     ZipCodeRecord(int zip, const std::string &place, const std::string &st,
00052                   const std::string &cnty, double lat, double lon)
00053         : zipCode(zip), placeName(place), state(st), county(cnty),
00054           latitude(lat), longitude(lon) {}
00055
00062     size_t getPackedSize() const
00063     {
00064         size_t size = 0;
00065         size += sizeof(int);            // zipCode
00066         size += placeName.length() + 1; // placeName + null
00067         size += state.length() + 1;     // state + null
00068         size += county.length() + 1;    // county + null
00069         size += sizeof(double);         // latitude
00070         size += sizeof(double);         // longitude
00071         return size;
00072     }
00073
00080     void display() const
00081     {
00082         std::cout « std::setfill(' ');
00083         std::cout « "  ZIP Code:   " « zipCode « std::endl;
```

```
        std::cout « "  Place:      " « placeName « std::endl;
        std::cout « "  State:      " « state « std::endl;
        std::cout « "  County:     " « county « std::endl;
        std::cout « "  Latitude:   " « latitude « std::endl;
        std::cout « "  Longitude:  " « longitude « std::endl;
    }

    void displayInline() const
    {
        std::cout « zipCode « "," « placeName « "," « state « ","
                  « county « "," « latitude « "," « longitude;
    }
};

#endif // ZIPCODERECORD_H
```

Index

~HeaderBuffer
  HeaderBuffer, 33
~LIBuffer
  LIBuffer, 38

activeListHead
  BlockedFileHeader, 13
add_seqset.cpp, 53
  addOneRecord, 54
  loadAddRecords, 54
  main, 54
  parseAddLine, 55
  printRecord, 55
  rebuildBlockIndex, 55
addEntry
  BlockIndex, 22
addField
  DataFileHeader, 27
addOneRecord
  add_seqset.cpp, 54
availListHead
  BlockedFileHeader, 13

BlockBuffer, 5
  BlockBuffer, 7
  canFit, 7
  clearToAvail, 7
  findRecord, 7
  getBlockSize, 8
  getFreeBytes, 8
  getHighestKey, 8
  getNextRBN, 8
  getPrevRBN, 9
  getRecordCount, 9
  getRecords, 9
  getUsedBytes, 9
  insertRecordSorted, 9
  isActiveBlock, 10
  isAvailBlock, 10
  pack, 10
  removeRecordByZip, 10
  setNextRBN, 11
  setPrevRBN, 11
  unpack, 11
BlockBuffer.cpp, 56
BlockBuffer.h, 56
blockCount
  BlockedFileHeader, 13
BlockedFileHeader, 12
  activeListHead, 13
  availListHead, 13
  blockCount, 13
  blockSize, 13
  fieldCount, 13
  fields, 14
  fileStructureType, 14
  headerSize, 14
  indexFileStructureType, 14
  minBlockCapacity, 14
  primaryKeyFieldIndex, 14
  primaryKeyIndexFileName, 14
  recordCount, 14
  recordSizeIntegerBytes, 15
  sizeFormatType, 15
  sizeInclusionFlag, 15
  sizeOfSizes, 15
  staleFlag, 15
  version, 15
BlockedFileHeader.cpp, 58
BlockedFileHeader.h, 59
BlockFileBuffer, 16
  BlockFileBuffer, 17
  getBlockOffset, 17
  getFilename, 17
  isReadOpen, 17
  isWriteOpen, 17
  openForRead, 18
  openForUpdate, 18
  openForWrite, 18
  readBlock, 18
  readHeader, 19
  writeBlock, 19
  writeHeader, 20
BlockFileBuffer.cpp, 60
BlockFileBuffer.h, 61
BlockIndex, 20
  addEntry, 22
  BlockIndex, 21
  findCandidateBlock, 22
  getEntries, 22
  getFilename, 22
  loadIndex, 23
  size, 23
  writeIndex, 23
BlockIndex.cpp, 62
BlockIndex.h, 63
BlockIndexEntry, 24
  maxKey, 24
  rbn, 24

blockSize
BlockedFileHeader, 13
build_block_index.cpp, 64
build_seqset.cpp, 65
buildPayload
RecordBuffer, 47

canFit
BlockBuffer, 7
clearToAvail
BlockBuffer, 7
close
LIBuffer, 39

DataFileHeader, 24
addField, 27
DataFileHeader, 26
display, 27
fieldCount, 29
fields, 29
fileStructureType, 29
getPackedSize, 27
headerSize, 29
pack, 27
primaryKeyFieldIndex, 29
primaryKeyIndexFileName, 29
recordCount, 29
recordSizeIntegerBytes, 29
sizeFormatType, 30
sizeInclusionFlag, 30
sizeOfSizes, 30
unpack, 28
version, 30
DataFileHeader.cpp, 66
DataFileHeader.h, 66
delete_seqset.cpp, 67
display
DataFileHeader, 27
ZipCodeRecord, 52
displayInline
ZipCodeRecord, 52
dump_seqset.cpp, 68

fieldCount
BlockedFileHeader, 13
DataFileHeader, 29
FieldDescriptor, 30
FieldDescriptor, 31
isPrimaryKey, 32
name, 32
typeSchema, 32
fields
BlockedFileHeader, 14
DataFileHeader, 29
fileStructureType
BlockedFileHeader, 14
DataFileHeader, 29
findCandidateBlock
BlockIndex, 22
findRecord
BlockBuffer, 7

getBlockOffset
BlockFileBuffer, 17
getBlockSize
BlockBuffer, 8
getCurrentOffset
LIBuffer, 39
getEntries
BlockIndex, 22
getFilename
BlockFileBuffer, 17
BlockIndex, 22
LIBuffer, 39
getFreeBytes
BlockBuffer, 8
getHighestKey
BlockBuffer, 8
getNextRBN
BlockBuffer, 8
getNextRecord
LIBuffer, 40
getPackedSize
DataFileHeader, 27
RecordBuffer, 47
ZipCodeRecord, 52
getPrevRBN
BlockBuffer, 9
getRecordCount
BlockBuffer, 9
LIBuffer, 40
getRecords
BlockBuffer, 9
getUsedBytes
BlockBuffer, 9

HeaderBuffer, 32
~HeaderBuffer, 33
HeaderBuffer, 33
readHeader, 34
writeHeader, 34
HeaderBuffer.cpp, 69
HeaderBuffer.h, 70
headerSize
BlockedFileHeader, 14
DataFileHeader, 29

indexFileStructureType
BlockedFileHeader, 14
insertRecordSorted
BlockBuffer, 9
isActiveBlock
BlockBuffer, 10
isAvailBlock
BlockBuffer, 10
isPrimaryKey
FieldDescriptor, 32
isReadOpen

BlockFileBuffer, 17
isWriteOpen
BlockFileBuffer, 17

LIBuffer, 36
~LIBuffer, 38
close, 39
getCurrentOffset, 39
getFilename, 39
getNextRecord, 40
getRecordCount, 40
LIBuffer, 37
openForRead, 41
openForWrite, 41
readHeader, 42
readRecordAt, 43
writeHeader, 44
writeRecord, 45
LIBuffer.cpp, 71
LIBuffer.h, 71
loadAddRecords
add_seqset.cpp, 54
loadIndex
BlockIndex, 23

main
add_seqset.cpp, 54
maxKey
BlockIndexEntry, 24
minBlockCapacity
BlockedFileHeader, 14

name
FieldDescriptor, 32

openForRead
BlockFileBuffer, 18
LIBuffer, 41
openForUpdate
BlockFileBuffer, 18
openForWrite
BlockFileBuffer, 18
LIBuffer, 41

pack
BlockBuffer, 10
DataFileHeader, 27
RecordBuffer, 48
parseAddLine
add_seqset.cpp, 55
primaryKeyFieldIndex
BlockedFileHeader, 14
DataFileHeader, 29
primaryKeyIndexFileName
BlockedFileHeader, 14
DataFileHeader, 29
printRecord
add_seqset.cpp, 55

rbn
BlockIndexEntry, 24
readBlock
BlockFileBuffer, 18
readHeader
BlockFileBuffer, 19
HeaderBuffer, 34
LIBuffer, 42
readLength
RecordBuffer, 48
readRecordAt
LIBuffer, 43
rebuildBlockIndex
add_seqset.cpp, 55
RecordBuffer, 46
buildPayload, 47
getPackedSize, 47
pack, 48
readLength, 48
unpack, 49
writeLength, 49
RecordBuffer.cpp, 73
RecordBuffer.h, 74
recordCount
BlockedFileHeader, 14
DataFileHeader, 29
recordSizeIntegerBytes
BlockedFileHeader, 15
DataFileHeader, 29
removeRecordByZip
BlockBuffer, 10

search_seqset.cpp, 75
setNextRBN
BlockBuffer, 11
setPrevRBN
BlockBuffer, 11
size
BlockIndex, 23
sizeFormatType
BlockedFileHeader, 15
DataFileHeader, 30
sizeInclusionFlag
BlockedFileHeader, 15
DataFileHeader, 30
sizeOfSizes
BlockedFileHeader, 15
DataFileHeader, 30
staleFlag
BlockedFileHeader, 15

typeSchema
FieldDescriptor, 32

unpack
BlockBuffer, 11
DataFileHeader, 28
RecordBuffer, 49

version

BlockedFileHeader, 15
DataFileHeader, 30

writeBlock
BlockFileBuffer, 19
writeHeader
BlockFileBuffer, 20
HeaderBuffer, 34
LIBuffer, 44
writeIndex
BlockIndex, 23
writeLength
RecordBuffer, 49
writeRecord
LIBuffer, 45

ZipCodeBuffer.cpp, 76
ZipCodeRecord, 50
display, 52
displayInline, 52
getPackedSize, 52
ZipCodeRecord, 51
ZipCodeRecord.h, 76